CO5 AT2-ASSIGNMENT1

SUTDENT NAME: P. VISHNU VARDHAN REDDY

REG. NO: 192425212

COURSE CODE: DSA0108

Q1. Develop a Banking Management System that supports Savings, Current, and Loan Accounts. The system must allow users to perform operations such as account creation, deposit, withdrawal, balance enquiry, and loan processing.(CO5) Requirements:

1. Define an abstract base class Account with virtual and pure virtual functions for common operations (deposit, withdraw, displayBalance, getAccountType).

2. Use runtime polymorphism (base class pointers/references) so the correct accountspecific behavior executes based on the actual object type.

3. Handle errors using exception handling — insufficient balance, invalid account number, negative deposit, and invalid withdrawal.

4. Demonstrate throwing, catching, and rethrowing exceptions during a transaction (e.g., a failed fund transfer).

5.Use exception specifications (throw(...)) on functions to declare the exceptions they may raise.

Aim

To develop a Banking Management System in C++ using abstract classes, virtual functions, runtime polymorphism, and exception handling to manage Savings, Current, and Loan accounts with operations such as account creation, deposit, withdrawal, balance enquiry, and fund transfer.

Algorithm

1. Start the program.

2. Define an abstract base class Account with pure virtual functions:
    - deposit()
    - withdraw()
    - displayBalance()
    - getAccountType()
3. Create derived classes:
    - SavingsAccount
    - CurrentAccount
    - LoanAccount
4. Override the virtual functions in each derived class.
5. Create account objects and store them using **base-class pointers** to demonstrate runtime polymorphism.
6. Implement exception handling for:
    - Invalid account number
    - Negative deposit
    - Invalid withdrawal
    - Insufficient balance
7. Implement fund transfer using try, catch, throw, and **rethrowing** of exceptions.
8. Use dynamic binding so the appropriate account operation is executed.
9. Display account details and balances.
10. Stop the program.

Code:

```
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;

// Abstract Base Class
class Account {
protected:
int accNo;
double balance;

public:
Account(int no, double bal) {
accNo = no;
balance = bal;
}

virtual void deposit(double amount) = 0;
virtual void withdraw(double amount) = 0;
virtual void displayBalance() = 0;
virtual string getAccountType() = 0;

int getAccountNo() {
return accNo;
}

virtual ~Account() {}
```

```
};

// Savings Account
class SavingsAccount : public Account {
public:
SavingsAccount(int no, double bal) : Account(no, bal) {}

void deposit(double amount) override {
if (amount <= 0)
throw invalid_argument("Negative/invalid deposit!");
balance += amount;
}

void withdraw(double amount) override {
if (amount <= 0)
throw invalid_argument("Invalid withdrawal!");
if (amount > balance)
throw runtime_error("Insufficient balance!");
balance -= amount;
}

void displayBalance() override {
cout << "Savings Account Balance: " << balance << endl;
}

string getAccountType() override {
return "Savings";
}
};

// Current Account
class CurrentAccount : public Account {
public:
CurrentAccount(int no, double bal) : Account(no, bal) {}

void deposit(double amount) override {
if (amount <= 0)
throw invalid_argument("Negative/invalid deposit!");
balance += amount;
}

void withdraw(double amount) override {
if (amount <= 0)
throw invalid_argument("Invalid withdrawal!");
if (amount > balance)
throw runtime_error("Insufficient balance!");
balance -= amount;
}

void displayBalance() override {
cout << "Current Account Balance: " << balance << endl;
}

string getAccountType() override {
return "Current";
}
};
```

```
// Loan Account
class LoanAccount : public Account {
public:
LoanAccount(int no, double loan) : Account(no, loan) {}

void deposit(double amount) override {
if (amount <= 0)
throw invalid_argument("Negative/invalid payment!");
balance -= amount;
if (balance < 0)
balance = 0;
}

void withdraw(double amount) override {
throw runtime_error("Withdrawal not allowed from Loan Account!");
}

void displayBalance() override {
cout << "Loan Amount Due: " << balance << endl;
}

string getAccountType() override {
return "Loan";
}
};

// Fund Transfer Function
void transfer(Account *from, Account *to, double amount) {
try {
if (from == nullptr || to == nullptr)
throw invalid_argument("Invalid account number!");

from->withdraw(amount);
to->deposit(amount);

cout << "Transfer successful!" << endl;
}
catch (const exception &e) {
cout << "Transaction Failed: " << e.what() << endl;
throw; // Rethrowing exception
}
}

int main() {
try {
// Runtime Polymorphism
Account *a1 = new SavingsAccount(101, 5000);
Account *a2 = new CurrentAccount(102, 10000);
Account *a3 = new LoanAccount(103, 20000);

cout << "===== ACCOUNT DETAILS =====" << endl;

cout << a1->getAccountType() << " Account" << endl;
a1->displayBalance();

cout << a2->getAccountType() << " Account" << endl;
```

```
a2->displayBalance();

cout << a3->getAccountType() << " Account" << endl;
a3->displayBalance();

cout << "\n===== DEPOSIT =====" << endl;
a1->deposit(2000);
a1->displayBalance();

cout << "\n===== WITHDRAW =====" << endl;
a1->withdraw(1000);
a1->displayBalance();

cout << "\n===== FUND TRANSFER =====" << endl;
try {
transfer(a1, a2, 2000);
}
catch (const exception &e) {
cout << "Main caught rethrown exception: "
<< e.what() << endl;
}

cout << "\n===== INVALID TRANSACTION =====" << endl;
try {
a1->withdraw(100000);
}
catch (const exception &e) {
cout << "Exception caught: " << e.what() << endl;
}

cout << "\n===== FINAL BALANCES =====" << endl;
a1->displayBalance();
a2->displayBalance();
a3->displayBalance();

delete a1;
delete a2;
delete a3;
}
catch (const exception &e) {
cout << "Exception: " << e.what() << endl;
}

return 0;
}
```

Output:

```
===== ACCOUNT DETAILS =====
Savings Account
Savings Account Balance: 5000
Current Account
Current Account Balance: 10000
Loan Account
Loan Amount Due: 20000

===== DEPOSIT =====
Savings Account Balance: 7000

===== WITHDRAW =====
Savings Account Balance: 6000

===== FUND TRANSFER =====
Transfer successful!

===== INVALID TRANSACTION =====
Exception caught: Insufficient balance!

===== INVALID TRANSACTION =====
Exception caught: Insufficient balance!

===== FINAL BALANCES =====
Savings Account Balance: 4000
Current Account Balance: 12000
Loan Amount Due: 20000
```

2. Develop an Online Payment System for an e-commerce application that supports Credit Card, Debit Card, UPI, and Net Banking payments. The system should process different payment methods and handle transaction failures safely.(CO5) Requirements:

1. Define an abstract base class PaymentMethod with virtual and pure virtual functions forming a common payment interface (validateDetails, pay, getMethodName).

2. Use runtime polymorphism to dynamically select and execute the correct payment method's logic at runtime.

3. Handle errors using exception handling — invalid card details, insufficient funds, incorrect UPI PIN, and failed transactions.

4. Demonstrate throwing, catching, and rethrowing exceptions during payment processing (e.g., in a payment gateway's checkout flow).

1. Use exception specifications (throw(...)) to declare possible payment-related exceptions on each function. 6. Display appropriate success or error messages based on the transaction outcome.

Aim

To develop an **Online Payment System in C++** using an abstract class, virtual functions, runtime polymorphism, and exception handling to support **Credit Card, Debit Card, UPI, and Net Banking** payments and safely handle transaction failures.

Algorithm

1. Start the program.
2. Define an abstract base class PaymentMethod.
3. Declare pure virtual functions:
    - validateDetails()
    - pay()
    - getMethodName()
4. Create derived classes:
    - CreditCard
    - DebitCard
    - UPI
    - NetBanking

5. Override the virtual functions in each derived class.
6. Create payment objects using **base-class pointers**.
7. Validate the payment details.
8. Process the payment according to the selected payment method.
9. Throw exceptions for:
    - Invalid card details
    - Insufficient funds
    - Incorrect UPI PIN
    - Failed transactions
10. Use try-catch blocks to catch payment exceptions.
11. Rethrow exceptions from the payment gateway to demonstrate exception propagation.
12. Display the payment status.
13. Stop the program.

Code :

```
#include
#include
#include
using namespace std;
class PaymentMethod{
public:
virtual void validateDetails()=0;
virtual void pay(double amount)=0;
virtual string getMethodName()=0;
virtual ~PaymentMethod(){}
};
class CreditCard:public PaymentMethod{
string cardNo;
double limit;
public:
CreditCard(string c,double l){cardNo=c;limit=l;}
void validateDetails()override{
if(cardNo.length()!=16)
throw invalid_argument("Invalid credit card details!");
}
void pay(double amount)override{
validateDetails();
if(amount>limit)
throw runtime_error("Credit card limit exceeded!");
cout<<"Credit Card payment successful: Rs."<<amount<<endl;
}
string getMethodName()override{return "Credit Card";}
};
```

```
class DebitCard:public PaymentMethod{
string cardNo;
double balance;
public:
DebitCard(string c,double b){cardNo=c;balance=b;}
void validateDetails()override{
if(cardNo.length()!=16)
throw invalid_argument("Invalid debit card details!");
}
void pay(double amount)override{
validateDetails();
if(amount>balance)
throw runtime_error("Insufficient funds!");
balance-=amount;
cout<<"Debit Card payment successful: Rs."<<amount<<endl;
}
string getMethodName()override{return "Debit Card";}
};
class UPI:public PaymentMethod{
string upiId;
int pin;
public:
UPI(string id,int p){upiId=id;pin=p;}
void validateDetails()override{
if(upiId.find("@")==string::npos)
throw invalid_argument("Invalid UPI ID!");
}
void pay(double amount)override{
validateDetails();
if(pin!=1234)
throw runtime_error("Incorrect UPI PIN!");
cout<<"UPI payment successful: Rs."<<amount<<endl;
}
string getMethodName()override{return "UPI";}
};
class NetBanking:public PaymentMethod{
string username;
string password;
public:
NetBanking(string u,string p){username=u;password=p;}
void validateDetails()override{
if(username.empty()||password.empty())
throw invalid_argument("Invalid Net Banking details!");
}
void pay(double amount)override{
validateDetails();
if(amount<=0)
```

```
throw runtime_error("Transaction failed!");
cout<<"Net Banking payment successful: Rs."<<amount<<endl;
}
string getMethodName()override{return "Net Banking";}
};
void checkout(PaymentMethodmethod,double amount){
try{
cout<<"\nProcessing using "<getMethodName()<<"..."<<endl;
method->validateDetails();
method->pay(amount);
}
catch(const exception&e){
cout<<"Gateway Error: "<<e.what()<<endl;
throw;
}
}
int main(){
PaymentMethodpayment;
payment=new DebitCard("1234567890123456",5000);
try{
checkout(payment,2000);
}
catch(const exception&e){
cout<<"Main caught exception: "<<e.what()<<endl;
}
delete payment;
cout<<"\n--- UPI Payment ---"<<endl;
payment=new UPI("user@upi",1234);
try{
checkout(payment,1000);
}
catch(const exception&e){
cout<<"Main caught exception: "<<e.what()<<endl;
}
delete payment;
cout<<"\n--- Invalid Payment ---"<<endl;
payment=new UPI("user@upi",1111);
try{
checkout(payment,500);
}
catch(const exception&e){
cout<<"Main caught rethrown exception: "<<e.what()<<endl;
}
delete payment;
return 0;
}
```

Output:

```
Processing using Debit Card...
Debit Card payment successful: Rs.2000

--- UPI Payment ---

Processing using UPI...
UPI payment successful: Rs.1000

--- Invalid Payment ---

Processing using UPI...
Gateway Error: Incorrect UPI PIN!
Main caught rethrown exception: Incorrect UPI PIN!
```